1. Linux operating system

What's an OS

- Resource allocator - manages cpu and
- Controller

Multiprogramming - Multitasking / Timeslicing

- Operative sys switches from different process to keep the CPU busy
- Utilizes timeslicing to incrementally progress on all processes
- Linux is the 4th generation of operative systems (1991).
- 5th gen - Virtualization: VMs hypervizors and containers

Kernel structures

- Monolithic - linux - runs everything - fast - vulnerable
- Hybrid - Windows - microkernel wrapped in monolithic layers.

Operating system structure:

User applications	Your own programs
System call interface	your programs access portal to system
Kernel	controller of system programs/services
System programs/services	File system, memory management, I/O, etc

Dual mode operation:

- Usermode and kernel mode - mode bit 1/0
- separates the userprocesses from the kernel

2. Linux processes

Linux processes – Understanding the basic process functions

In Linux, a process is simply a program that is currently running. Linux provides several functions (called system calls) that allow programs to create processes, execute other programs, wait for processes to finish, and terminate them.

The most important functions to understand are:

- `fork()` – Creates a new process.
- `exec()` – Replaces the current process with another program.
- `wait()` / `waitpid()` – Waits for a child process.
- `exit()` – Terminates a process.
- `getpid()` / `getppid()` – Retrieves process IDs.

Every process has several important properties:

Property	Meaning
PID	Process ID – unique identifier for the running process
PPID	Parent Process ID – identifies the process that created it
Memory	The process's own virtual address space
State	Running, sleeping, stopped, zombie, etc.
Resources	Open files, file descriptors, and other resources

Linux processes form a parent-child hierarchy. A process can create another process, which becomes its child.

1. `fork()` – Creating a child process

The `fork()` function is one of the most fundamental concepts in Linux.

When a process calls `fork()`, Linux creates a new process that is initially a copy of the original process.

- The original process is called the parent.
- The newly created process is called the child.
- Both continue executing from the instruction immediately after `fork()`.

How can we distinguish the parent from the child?

The key is the return value of `fork()`:

Return value	Meaning
`0`	You are in the child process
Positive integer	You are in the parent process; the value is the child's PID
`-1`	Fork failed; no child was created

Important: Both processes have their own `pid` variable, but `fork()` returns a different value to each.

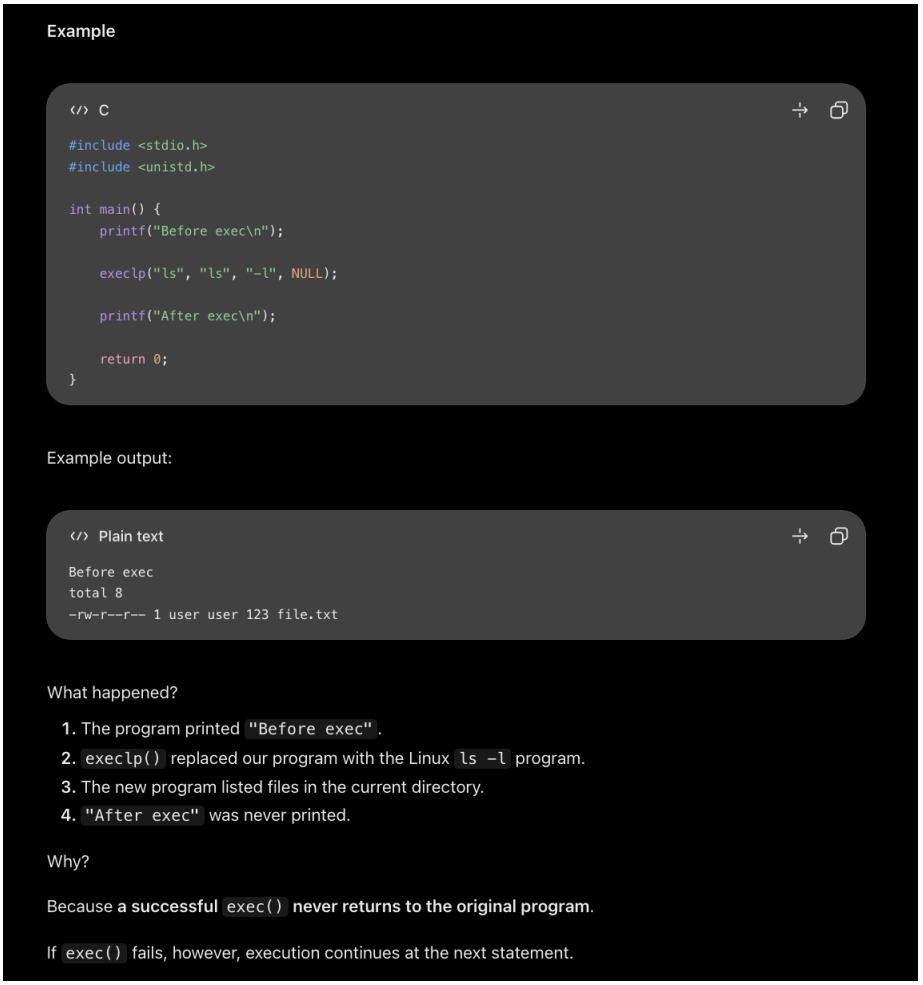
2. `exec()` – Running another program

The `exec()` family of functions replaces the currently executing program with another program.

This is different from `fork()`:

- `fork()` creates a new process.
- `exec()` replaces the program running inside an existing process.

The process keeps its PID after a successful `exec()`.



3. `wait()` – Waiting for a child process

When you create a child using `fork()`, the parent and child can run concurrently.

Sometimes we want the parent to wait until the child finishes before continuing.

That's what `wait()` does.

The `wait(NULL)` statement blocks the parent until a child terminates. It also collects the child's termination status, preventing it from remaining a zombie process.

`wait()` versus `waitpid()`

`wait()` waits for any one child, while `waitpid()` allows you to specify which child to wait for.

4. `exit()` – Terminating a process

The `exit()` function terminates the calling process.

5. `getpid()` and `getppid()` – Identifying processes

Every running process has a PID (process ID).

Two useful functions are:

- `getpid()` returns the PID of the current process.
- `getppid()` returns the PID of its parent.

6. Additional concepts: zombie and orphan processes

Zombie process: A child has terminated, but its parent has not yet collected its termination status using `wait()` or `waitpid()`. The child no longer executes code, but a small process-table entry remains until it is reaped.

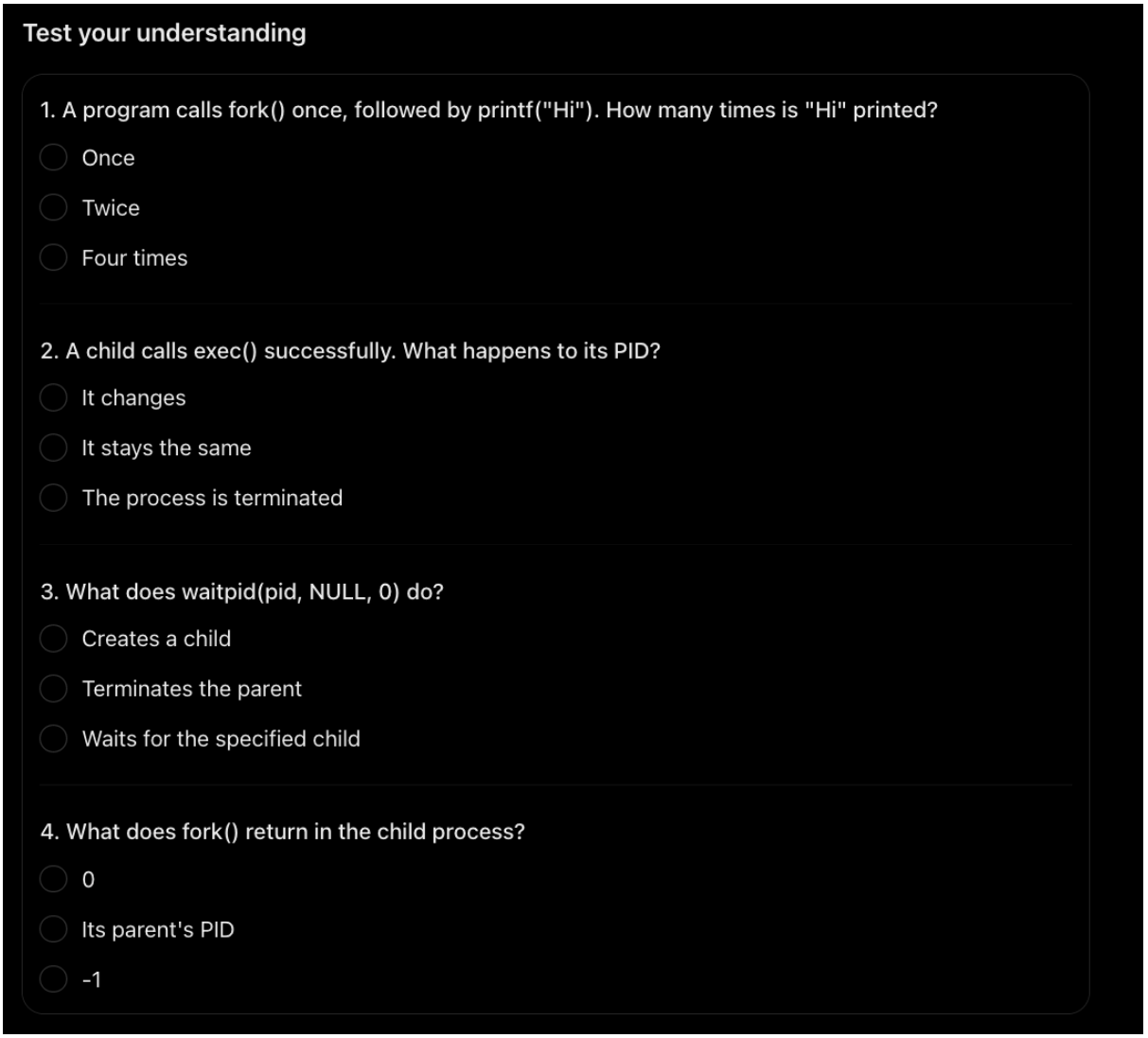
Orphan process: A child is still running when its parent terminates. Linux reassigns the child to another process, typically PID 1 or a designated subreaper.

Neither concept means the process is necessarily malfunctioning.

Summary: The five most important functions

Function	Purpose	Think of it as...
`fork()`	Creates a child process	"Make a copy of me"
`exec()`	Replaces the running program	"Run this program instead"
`wait()`	Waits for a child to finish	"Don't continue until a child finishes"
`exit()`	Terminates a process	"I'm done"
`getpid()`	Retrieves the current process ID	"Who am I?"

The most important distinction is that `fork()` creates a new process, while `exec()` changes which program a process runs. Together with `wait()`, these functions form the foundation of process management in Linux.



3. IPC

Inter-Process Communication (IPC) in Linux

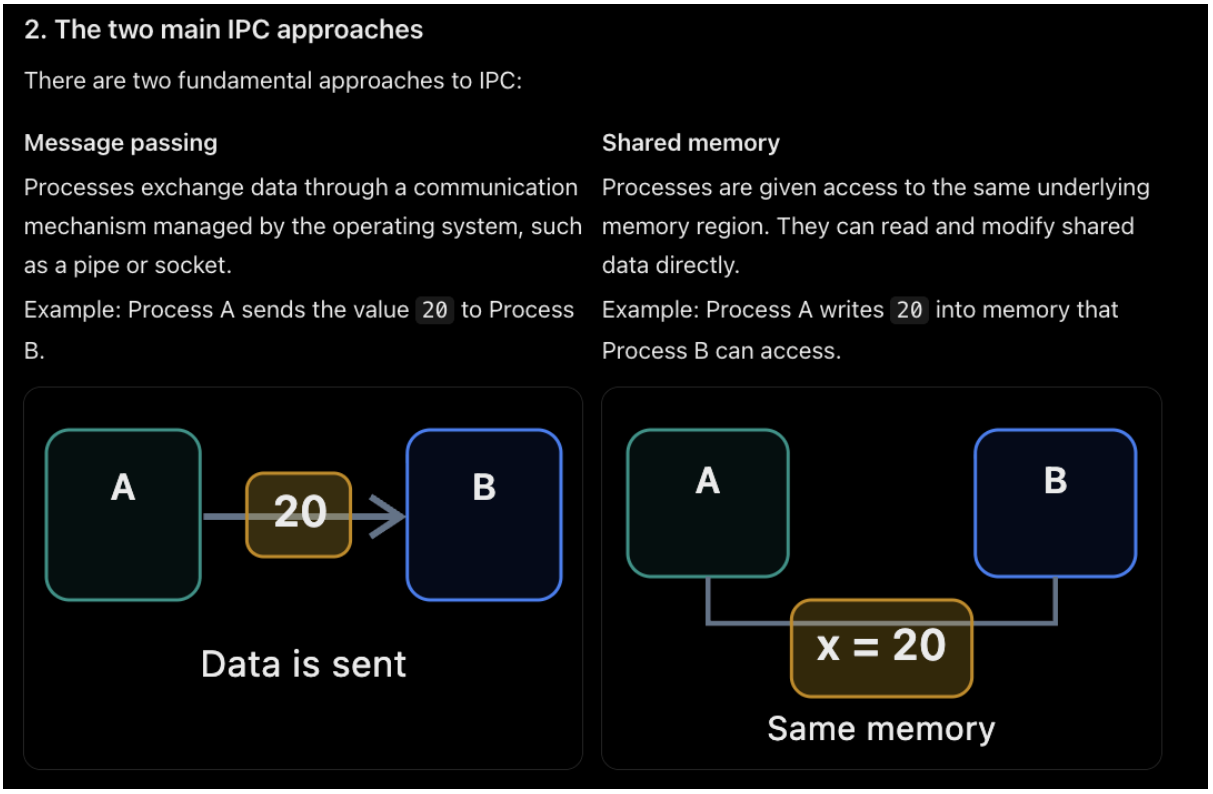
IPC (Inter-Process Communication) refers to the mechanisms that allow different processes to communicate and coordinate with each other.

To understand IPC, remember what happens when we use `fork()`:

- A parent process creates a child process.
- The child initially has a copy of the parent's memory.
- However, they are separate processes with separate virtual address spaces.

- If the child changes a normal variable, that change is not visible to the parent.

IPC provides ways for these processes to exchange information, share data, and synchronize their execution.



3. Pipes – Sending data between processes

A pipe is one of the simplest IPC mechanisms.

Think of it as a tube:

- One process puts data into one end.
- Another process reads data from the other end.
- Data travels in one direction through a conventional pipe.

Basic pipe functions

Function	Purpose
`pipe(fd)`	Creates a pipe with two file descriptors
`write()`	Writes data into the pipe

`read()` Reads data from the pipe

`close()` Closes a pipe endpoint

When calling `pipe(fd)`, Linux fills two integers:

- `fd[0]`: the reading end.
- `fd[1]`: the writing end.

Notice that `read()` can block if the pipe is empty and a writer is still open. In the example, this allows the child to wait until data arrives.

Important: A pipe transports bytes, not C variables or complete messages automatically. Programs must agree on how those bytes should be interpreted.

4. Shared memory – Multiple processes accessing the same memory

Normally, each process has isolated memory. But Linux can map the same underlying memory into multiple processes.

Linux functions for shared memory

Linux provides different APIs for this, including:

- `mmap()` – Maps memory, including regions shared between processes.
- `shm_open()` – Creates or opens a POSIX shared-memory object.
- `munmap()` – Removes a memory mapping.

Shared memory can be very fast because processes do not need to send every piece of data through a pipe.

5. Synchronization – Preventing processes from interfering

IPC isn't only about exchanging data. Sometimes processes must coordinate when they access shared resources.

Semaphores

A semaphore is a synchronization mechanism that controls access to a resource.

A binary semaphore can be used similarly to a lock:

- `wait()` / `sem_wait()` – Acquire permission. If unavailable, wait.
- `post()` / `sem_post()` – Release permission, allowing another process to proceed.

Mutexes

A mutex (mutual exclusion lock) serves a similar purpose, allowing only one execution context to hold the lock at a time.

The difference is that a mutex is generally an ownership-based lock, while a semaphore is a counter-based synchronization primitive.

For Linux processes, a mutex must be configured for inter-process sharing, and its storage must be accessible to the participating processes.

6. Signals – Sending notifications to processes

Signals are another IPC mechanism. They are primarily used to notify a process that an event has occurred.

For example:

- `SIGINT` – Commonly sent when pressing Ctrl+C.
- `SIGTERM` – Requests that a process terminate.
- `SIGKILL` – Forces termination; cannot be caught or ignored.
- `SIGCHLD` – Notifies a parent when a child changes state, such as terminating.

You can send a signal with the `kill()` system call.

Despite the name, `kill()` doesn't necessarily kill a process. It sends a signal.

7. Sockets – Communication between processes or computers

Sockets provide a communication endpoint that processes can use to exchange data.

Unlike ordinary pipes, sockets can support communication:

- Between processes on the same computer.
- Between processes on different computers over a network.

8. Other common IPC mechanisms

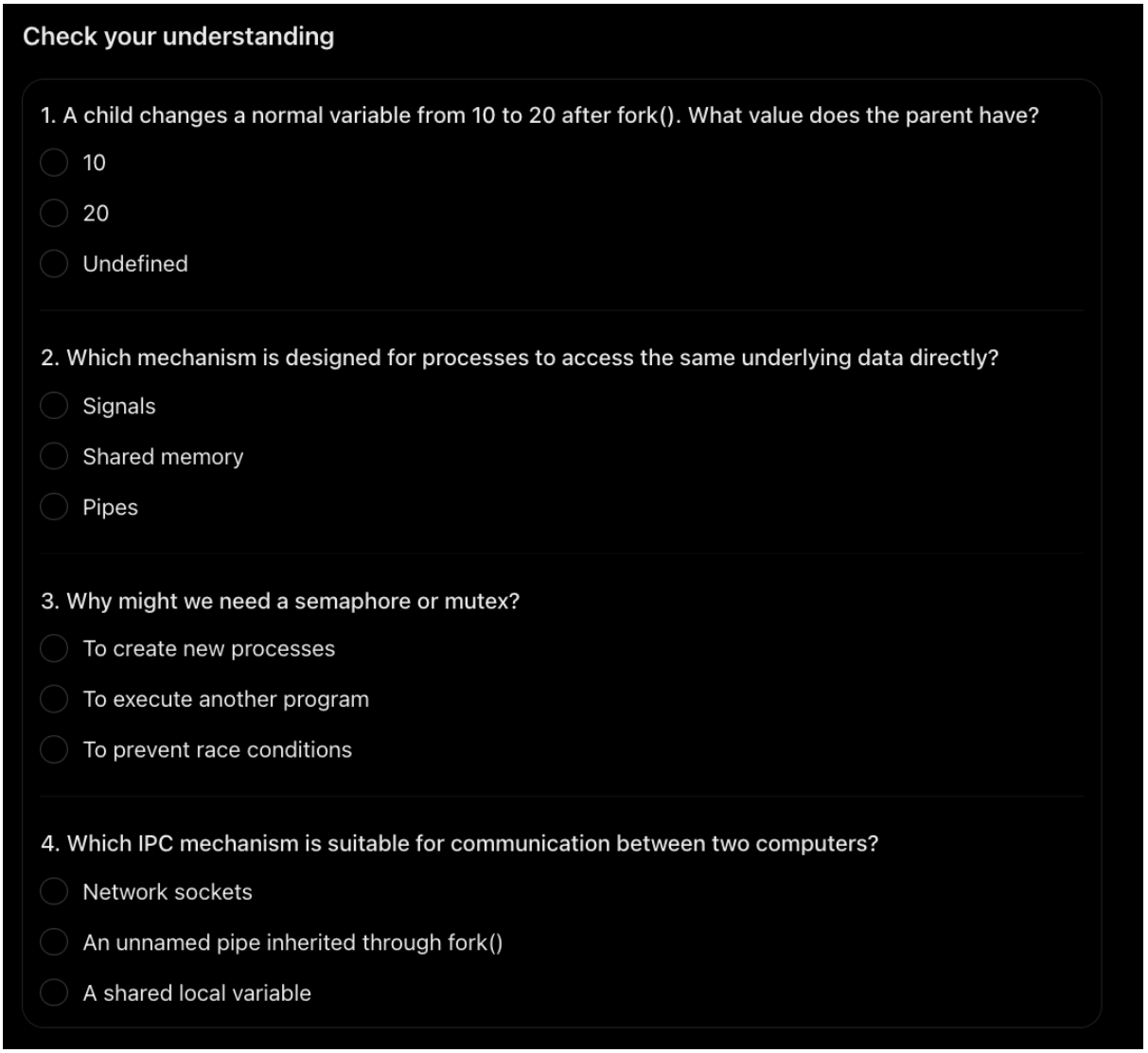
Two other concepts worth knowing are:

Named pipes (FIFOs): Similar to ordinary pipes, but identified by a filesystem path. They allow unrelated processes to communicate without needing to inherit descriptors from a common parent.

Message queues: Allow processes to send and receive separate messages through an operating-system-managed queue. Unlike an ordinary pipe's byte stream, message boundaries are preserved.

9. Comparing IPC mechanisms

Mechanism	Main purpose	Example
Pipes	One-way byte stream	Parent sends text to child
Named pipes	Byte stream between unrelated processes	Two terminal programs communicate
Shared memory	Directly share data	Two processes update shared information
Semaphores / mutexes	Synchronization	Prevent simultaneous modifications
Signals	Event notifications	Tell a process to terminate
Message queues	Exchange distinct messages	Workers receive tasks
Sockets	Local or network communication	Browser communicates with server



4. Scheduling

In a single-processor system only one process can run at a time, other processes must wait.

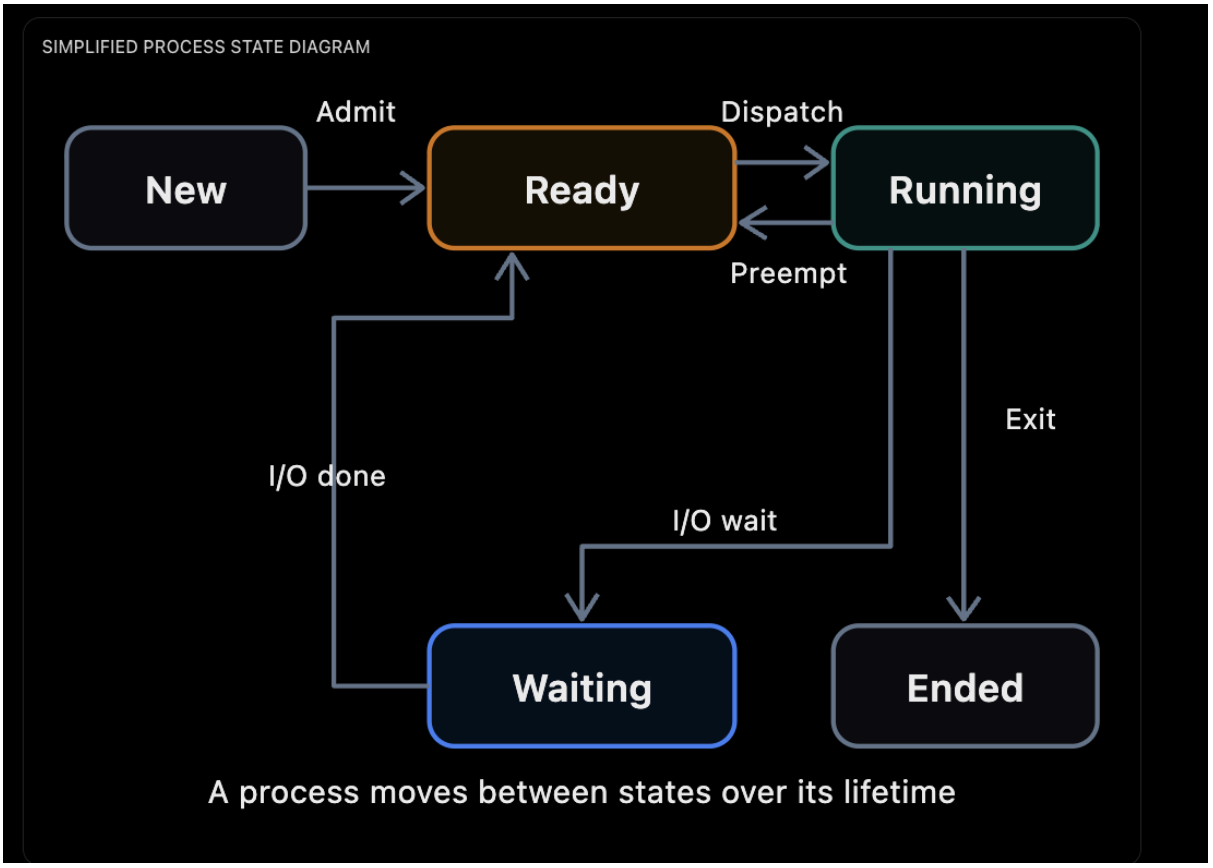
CPU-I/O burst cycle: A process execution consists of a cycle of CPU execution and I/O wait called CPU burst and I/O burst respectively. The I/O burst is where the processor waits for an I/O event, while a CPU burst is where the processor is busy, doing actual work.

Concurrency: small parts of multiple processes are executed after another. It appears to the user that all three programs are running simultaneously. (Can be done by only one cpu).

Parallelism: Multiple processes actually run simultaneously on different CPU cores. Needs more than one cpu core.

In a simplified operating-system model, a process can be in five states:

State	Meaning
New	The process is being created
Ready	Able to run, waiting for CPU time in queue
Running	Currently executing on a CPU, a process's state during CPU burst
Waiting / Blocked	Waiting for an event, such as I/O
Terminated	Finished executing



Example: A program reading a file

Consider:

```
None
read(fd, buffer, 100);
```

If the requested data isn't available immediately:

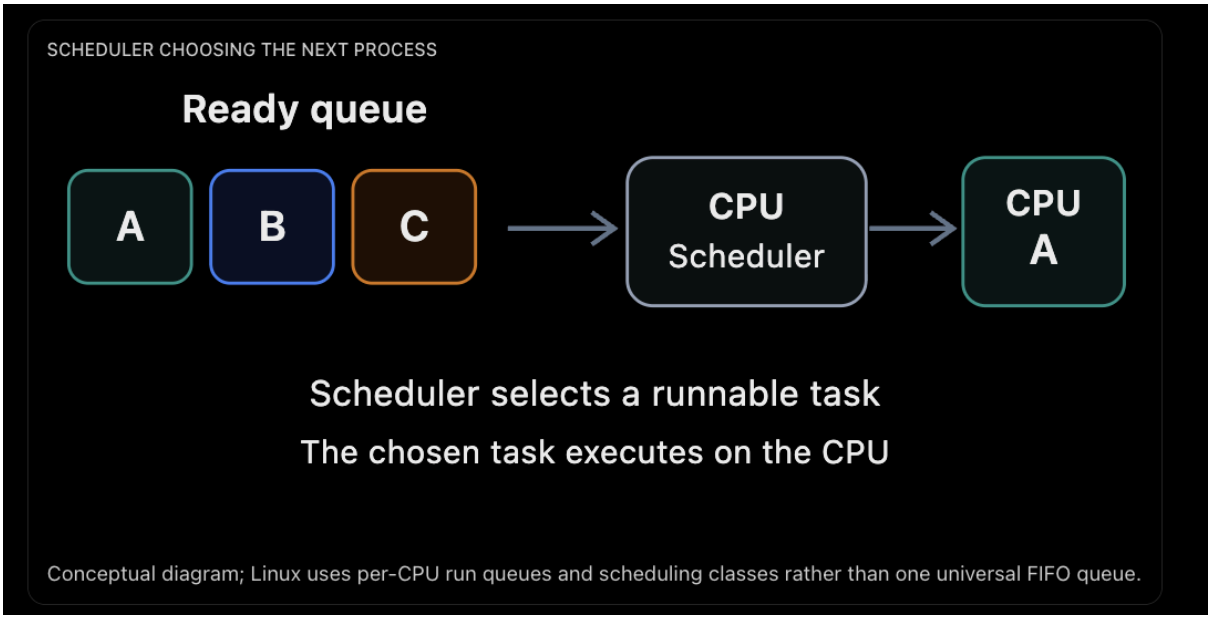
1. The running process requests the data.
2. It may enter a waiting state.
3. Linux schedules another ready process to use the CPU.
4. Once the data becomes available, the waiting process becomes ready again.
5. The scheduler eventually lets it run.

This is important: A process waiting for I/O does not normally consume CPU time while blocked.

3. The ready queue and the scheduler

The ready queue is a conceptual collection of processes that are able to execute but are not currently running.

Imagine the following situation:



The scheduler tries to achieve several goals:

- Fairness: Processes should receive an appropriate share of CPU time.
- Responsiveness: Interactive programs should react quickly.
- Throughput: Complete as much useful work as possible.
- Low waiting time: Reduce time spent waiting to run.
- Efficiency: Avoid unnecessary CPU switching overhead.

These goals sometimes conflict.

For example, frequent switching can improve responsiveness but also increase overhead.

4. Preemptive vs. non-preemptive scheduling

Non-preemptive scheduling

Once a process receives the CPU, it keeps executing until it finishes or voluntarily gives up the CPU, such as by blocking.

Preemptive scheduling

The operating system can interrupt a running process and schedule another process, even if the first process hasn't finished. All modern operating systems use preemptive scheduling.

A. First-Come, First-Served (FCFS)

Rule: Execute processes in arrival order. Once a process starts, it runs until its CPU burst ends.

Pros:

Simple

Cons:

Might cause a convoy effect: short tasks get stuck behind long tasks.

B. Shortest Job First (SJF)

Rule: Run the process with the shortest expected CPU burst first.

Pros:

Often reduces average waiting time

Cons:

The disadvantage is that the scheduler generally doesn't know exactly how long a process's next CPU burst will be.

Also, long-running processes could starve if shorter jobs keep arriving.

Possible solution:

CPU burst prediction, for example **Exponential average prediction:**

$$T_{n+1} = \alpha t_n + (1 - \alpha) T_n$$

- t_n = Den faktiska längden på den n-te CPU-bursten.
- T_n = Den förutsagda längden på den n-te CPU-bursten (börjar med random gissning för T_0)
- **α** = Utjämningsfaktor (smoothing factor), där 0 ≤ α ≤ 1.

C. Round Robin (RR)

Rule: Each process gets a limited amount of CPU time, called a time quantum.

Round Robin is **preemptive**, because a running process can be interrupted when its quantum expires.

The size of the quantum matters:

- If it's too large, Round Robin behaves similarly to FCFS.
- If it's too small, the computer spends more time switching between processes.
- Rule of thumb: Use time quantum that corresponds to about **80%** of the waiting process lengths (to both reduce overhead and avoid starvation)

D. Priority scheduling

In priority scheduling, the scheduler chooses processes based on priority.

Cons: Starvation: low-priority processes might wait indefinitely if higher-priority processes continuously need CPU time.

Possible solution: Aging, where waiting processes gradually gain priority. This is a general scheduling technique, not a universal rule used by every Linux policy.

7. Context switching

When the scheduler decides to stop running one process and start another, the CPU must change its execution context.

If the scheduler preempts Process A before it finishes, Linux must preserve enough execution state to resume it later.

This includes information such as:

- CPU registers.
- Program counter (instruction pointer).
- Stack pointer.
- Scheduling state.
- Memory-management context, when switching between address spaces.

The scheduler then restores the relevant state of Process B.

Context switching is essential for multitasking, but it isn't free.

The CPU spends time saving and restoring state to and from Process Control Blocks (PCB)'s instead of executing application instructions, this is known as the dispatch latency, which occurs every time a context switch happens. Switching between processes can also disturb CPU caches and memory-translation caches.

This explains why extremely small Round Robin time quanta can reduce efficiency.

8. CPU-bound and I/O-bound processes

CPU-bound process

Spends most of its time performing calculations.

Example: A program calculating millions of prime numbers or training a machine-learning model.

I/O-bound process

Frequently waits for input/output operations.

Example: A program reading files, waiting for network responses, or interacting with a user.

A good scheduler should accommodate both:

- Give CPU-bound processes opportunities to make progress.
- Allow I/O-bound interactive processes to respond promptly when they become runnable.

9. How scheduling actually works in modern Linux

The algorithms above are mainly teaching examples. Linux does not simply use FCFS or basic Round Robin for all normal processes.

Modern Linux uses several scheduling classes that handle different kinds of workloads.

Scheduling policy	Description
`SCHED_OTHER`	Default scheduling for ordinary processes
`SCHED_BATCH`	Intended for non-interactive CPU-intensive tasks

`SCHED_IDLE`	Very low-priority CPU scheduling
`SCHED_FIFO`	Real-time first-in, first-out scheduling
`SCHED_RR`	Real-time Round Robin scheduling
`SCHED_DEADLINE`	Deadline-based scheduling

EEVDF – Scheduling ordinary processes

Modern Linux kernels use a scheduling algorithm called EEVDF (Earliest Eligible Virtual Deadline First) for normal fair-scheduling tasks.

EEVDF selects among eligible tasks using virtual deadlines, with the aim of dividing CPU time fairly while supporting low latency.

It builds on concepts from the older Completely Fair Scheduler (CFS), which aimed to give processes a fair proportion of CPU time using virtual runtime.

The practical idea is:

- The scheduler tracks how much CPU service tasks have received relative to their weights.
- Tasks that have received less than their fair share become eligible for more CPU time.
- Virtual deadlines help determine which eligible task should run next.

Linux also uses priorities and task weights to influence how much CPU time processes receive.

10. Important scheduling measurements

Suppose a process:

- Arrives at time 0.
- Starts running at time 3.
- Finishes at time 10.
- Requires 5 ms total CPU time.

We can calculate:

Waiting time: Total time spent ready but not running.

$$W = T_{completion} - T_{arrival} - T_{burst}$$

$$W = 10 - 0 - 5 = 5ms$$

Turnaround time: Total time from arrival until completion.

$$T_{turnaround} = T_{completion} - T_{arrival}$$

$$T_{turnaround} = 10 - 0 = 10ms$$

Response time: Time between arrival and the first execution.

$$T_{response} = T_{firstrun} - T_{arrival}$$

$$T_{response} = 3 - 0 = 3ms$$

Notice that waiting time and response time are not necessarily the same. A process might run briefly, be preempted, and then wait again.

Test your understanding

1. A running process calls read() and must wait for data. What normally happens?

- ○ It keeps using the CPU while waiting
- ○ It becomes blocked and another process can run
- ○ It terminates

2. What does preemptive scheduling mean?

- ○ The OS can interrupt a running process
- ○ A process must finish before another runs
- ○ Processes always run in arrival order

3. With Round Robin scheduling, what happens when a process uses its quantum but still has work left?

- ○ It terminates
- ○ It gets another quantum immediately in all cases
- ○ It is returned to the ready queue

4. What is the turnaround time of a process arriving at 2 ms and finishing at 12 ms?

- ○ 10 ms
- ○ 12 ms
- ○ 14 ms

Memory